# PRODUCT REQUIREMENTS DOCUMENT

# Fraud Intelligence AI Assistant

*FIAA — Capstone Project 2025*

| | |
|---|---|
| **Project** | Fraud Intelligence AI Assistant (FIAA) |
| **Domain** | Banking — Fraud Detection & Reporting |
| **Version** | PRD v1.0 — 2025 |
| **Authors** | Darshan J · Sabi Qunnar |
| **Organisation** | EY GDS \| AI & Data Practice |
| **Document Type** | Product Requirements Document |
| **Status** | Final |

# 1. Project Definition

## 1.1 Overview

Fraud Intelligence AI Assistant (FIAA) is an autonomous multi-agent Generative AI system designed to automate fraud incident investigation and reporting in the banking domain. When a suspicious transaction is detected by the bank's fraud rule engine, FIAA automatically researches the case using live web intelligence and institutional knowledge, synthesises a complete fraud incident report with a risk score and recommended actions, and delivers it to the analyst's dashboard — all in under 90 seconds with zero manual input.

## 1.2 Problem Statement

*Fraud analysts manually spend 45–90 minutes per case — searching for fraud patterns, checking regulatory deadlines, pulling past cases, and writing reports. The 2-hour SLA for SWIFT recall decisions means slow investigation directly costs money that cannot be recovered.*

| Pain Point | Current State | Business Impact |
|---|---|---|
| Manual investigation time | 45–90 min per fraud case | Analyst throughput severely limited |
| SWIFT recall SLA | 2-hour decision window | Slow investigation = unrecoverable losses |
| Report writing | Fully manual — analyst writes from scratch | Inconsistent quality, high analyst fatigue |
| Regulatory deadlines | Manually looked up per case | Risk of missed RBI filings |
| Knowledge retrieval | Manual search of past cases & playbooks | Inconsistent, incomplete context |

## 1.3 Proposed Solution

FIAA replaces 45–90 minutes of manual research with a 90-second autonomous agent pipeline. The analyst's role becomes: read the report, click one action button.

| Before FIAA | After FIAA |
|---|---|
| 45–90 min investigation per case | 90-second automated pipeline |
| Manual web search for fraud patterns | Tavily live web search — automated |
| Manual search through past cases | ChromaDB RAG — instant retrieval |
| Analyst writes full report | Groq Llama 3.3 70B generates structured report |
| No quality check on reports | Evaluator + RAGAS quality gate |
| Manual risk decision | Scored 1–10 → auto-close or human review |

## 1.4 Project Type

| Attribute | Classification |
|---|---|
| Project category | Capstone / Proof-of-Concept — Production-Grade Architecture |
| Application type | Autonomous Multi-Agent AI System |

| Attribute | Classification |
|---|---|
| Deployment model | On-premise Docker Compose (4-service containerised stack) |
| Interface type | Web UI (Streamlit) + REST API (FastAPI webhook) |
| AI paradigm | RAG-augmented LLM + Multi-agent orchestration (LangGraph) |
| Domain | Banking — Fraud Detection & Incident Management |
| Regulatory context | RBI guidelines, SWIFT advisory compliance, PII handling |

## 1.5 Team

| Team Member | Role | Responsibilities |
|---|---|---|
| Darshan J | AI Engineer | Agent pipeline, RAG, LangGraph orchestration |
| Sabi Qunnar | AI Engineer | Guardrails, Observability, UI, Deployment |

# 2. Goals

## 2.1 Primary Goal

*Reduce fraud investigation time from 45–90 minutes to under 90 seconds by delivering a fully autonomous, evidence-grounded fraud incident report with risk score and recommended actions to the analyst's dashboard.*

## 2.2 Stage 1 Goals — Core Pipeline (MVP)

| Goal | Description | Success Metric |
|---|---|---|
| Webhook receiver | FastAPI endpoint that auto-accepts fraud alerts from bank rule engine | < 100ms response time to rule engine |
| Multi-agent pipeline | LangGraph Supervisor routing four research agents in sequence | All 4 agents complete per run |
| RAG knowledge base | ChromaDB with past fraud cases, RBI guidelines, playbooks | Top-5 relevant chunks retrieved per query |
| Report generation | Groq Llama 3.3 70B synthesising all context into structured fraud report | Full report generated in < 90 seconds |
| Evaluator quality gate | Verifies evidence, score, PII masking, RBI accuracy | Evaluator approval before delivery |
| Input & output guardrails | Injection detection, PII redaction, hallucination check | Zero PII in delivered report |
| Streamlit dashboard | Live agent status, report display, action buttons, download | Report visible within 90s of alert |

## 2.3 Stage 2 Goals — Production Hardening

| Goal | Description | Tool |
|---|---|---|
| Structured logging | JSON lines with run_id binding and secret redaction | structlog |
| Prometheus metrics | 7 metrics at /metrics for Grafana scraping | Prometheus + Grafana |
| LangSmith tracing | Full LangGraph chain traces with prompt/response replay | LangSmith + OpenTelemetry |
| RAGAS evaluation | Faithfulness, answer relevance, context recall scored per run | RAGAS |
| Docker Compose deployment | 4-service containerised deployment | Docker Compose |
| Auto-trigger modes | Manual fire, timer stream, random stream for demo | Streamlit + FastAPI |

## 2.4 Quantitative Success Criteria

| KPI | Target |
|---|---|
| End-to-end pipeline latency | < 90 seconds from webhook receipt to dashboard report |
| FastAPI response to rule engine | < 100 ms (background task — non-blocking) |
| RAGAS faithfulness score | >= 0.7 per run |
| Evaluator approval rate | >= 90% of runs on first attempt |
| RAG retrieval | Top-5 chunks per query, cosine similarity |
| Guardrail coverage | 100% of inputs and outputs scanned |
| PII in final report | Zero PII leakage post-redaction |

## 3. Non-Goals

The following are explicitly out of scope for this capstone project. They represent genuine technical decisions — not temporary gaps — and are documented to set clear expectations.

| Non-Goal | Reason / Boundary |
|---|---|
| Real bank system integration | FIAA simulates the webhook trigger. No live core banking connection, no production bank data. |
| Account freezing execution | FIAA recommends the action. The analyst executes manually via their banking system. |
| SWIFT recall execution | FIAA provides case details and timing. Correspondent banking team executes the recall. |
| Real-time Aadhaar / PAN validation | KYC document reading (PyMuPDF) is in scope; live government API verification is out of scope. |
| Training a custom LLM | Uses Groq-hosted Llama 3.3 70B via API. No fine-tuning or model training. |
| Multi-bank / multi-tenant deployment | Single bank instance. Multi-tenancy is future work. |
| Mobile application | Web-only Streamlit UI. No iOS / Android application. |
| Continuous transaction monitoring | Alert-based trigger only. Real-time streaming transaction analysis is future work. |
| Email / SMS notifications | Dashboard-only delivery. Push notifications are future work. |
| Core banking UI integration | Standalone Streamlit dashboard. Embedding into existing bank UI is future work. |

# 4. Feature Definitions

## Feature 1 — Automated Alert Ingestion (Webhook)

Priority: P0 — Critical | Owner: Backend / API layer

When the bank's fraud rule engine detects a suspicious transaction, it fires a POST request to FIAA's FastAPI endpoint automatically. The pipeline starts without any analyst involvement.

> *Design decision: FastAPI responds to the rule engine immediately (< 100ms) with {"status":"accepted"} and runs the pipeline as a background task. The rule engine is never kept waiting 90 seconds for the report.*

| Attribute | Detail |
|---|---|
| Endpoint | POST /api/analyse |
| Auth | X-API-Key header validation |
| Rate limiting | Per-IP rate limit to prevent flood |
| run_id | UUID assigned on receipt — bound to all logs and traces |
| Response | < 100ms — {"status": "accepted", "run_id": "..."} |
| Pipeline start | Background task — non-blocking |

### Trigger Modes

| Mode | Trigger | Use case |
|---|---|---|
| Production | Bank rule engine calls POST /api/analyse | Zero human involvement |
| Demo — Manual | Analyst clicks 'Fire Alert' in dashboard | Live demonstration |
| Demo — Timer | Auto-fires every 60 seconds | Hands-free presentation |
| Demo — Random | Random alert every 45 seconds | Mixed fraud alert simulation |

## Feature 2 — Multi-Agent Research Pipeline

Priority: P0 — Critical | Owner: AI Engineering

A LangGraph StateGraph orchestrates seven specialised agents. The Supervisor Agent uses LLM reasoning — not hardcoded rules — to route sub-tasks based on investigation state at every iteration.

| Agent | Responsibility | Model | Design Decision |
|---|---|---|---|
| Supervisor | LLM routing — reads state, decides next agent | Groq Llama 3.3 70B | LLM not rules — handles unseen alert types |
| Incident Agent | Parse, classify alert, log to database | Groq Llama 3.1 8B | Small model sufficient for extraction |
| Search Agent | Tavily live web — fraud patterns, RBI, SWIFT | Groq Llama 3.1 8B | Small model for synthesis; saves 70B quota |

| Agent | Responsibility | Model | Design Decision |
|---|---|---|---|
| RAG Agent | ChromaDB cosine — past cases, playbooks, rules | Local + Groq | Local embeddings — no customer data to external API |
| Reader Agent | Extract from uploaded KYC / account PDFs | Groq Llama 3.1 8B | Only runs if analyst uploads a document |
| Report Agent | Synthesise all context into fraud report | Groq Llama 3.3 70B | Large model reserved here — report quality critical |
| Evaluator | Quality gate — evidence, score, PII, RBI | Groq Llama 3.3 70B | Mirrors EY preparer-reviewer model |

### Supervisor Routing Logic

| Iteration | State | Decision | Reason |
|---|---|---|---|
| 0 | Everything missing | → SEARCH | Need live RBI + fraud corridor data |
| 1 | Search done, RAG missing | → RAG | Need past cases + playbooks |
| 2 | Search + RAG done | → WRITE | Sufficient context gathered |
| 3 | Evaluator approved | → END | Pipeline complete |
| 3 | Evaluator rejected | → WRITE | Revise with evaluator feedback |
| 5 | Iteration cap reached | → WRITE | Force report — prevent infinite loop |

## Feature 3 — RAG Knowledge Base

Priority: P0 — Critical | Owner: AI Engineering

| Step | Tool | Detail |
|---|---|---|
| 1. Ingest | rag/ingestor.py | fraud_playbooks.txt (312 chunks), rbi_guidelines.txt (428 chunks), past_cases.json (107 cases) |
| 2. Chunk | 512-token windows | 64-token overlap preserves context across chunk boundaries |
| 3. Embed | all-MiniLM-L6-v2 (local) | 384-dimension vectors — CPU only, no external API, no customer data leaves machine |
| 4. Store | ChromaDB PersistentClient | Cosine similarity index, persistent across restarts, metadata alongside each vector |
| 5. Retrieve | collection.query() | Top-5 chunks by cosine similarity to current alert embedding |
| 6. Augment | LangGraph state | Chunks injected into Report Agent prompt as structured context |

*Why ChromaDB: Runs locally (no cloud dependency), persists across restarts (FAISS loses index on Streamlit rerun), stores metadata alongside vectors, and is free. Pinecone sends data to cloud — banking compliance violation.*

## Feature 4 — Report Generation & Quality Gate

Priority: P0 — Critical | Owner: AI Engineering

The Report Agent (Groq Llama 3.3 70B) synthesises findings from all research agents into a structured fraud incident report. A dual quality gate — Evaluator Agent + RAGAS — verifies output before delivery.

| Report Section | Content |
|---|---|
| Risk score | 1–10 composite score weighted across all signals |
| Evidence table | Fraud signals with source citations |
| CRIS draft | Crime Reference Investigation Summary for regulatory filing |
| Recommended actions | Account freeze / SWIFT recall / enhanced monitoring / close |
| Timeline | Transaction chronology with anomaly flags |
| Regulatory deadlines | RBI filing windows, SWIFT recall window |

### Evaluator Checks

| Check | Criterion |
|---|---|
| Evidence grounding | Every claim in report traceable to a retrieved source |
| Score proportionality | Risk score consistent with number and severity of signals |
| PII masking | No raw account numbers, Aadhaar, PAN in output |
| RBI deadline accuracy | Filing deadlines accurate to RBI circulars in knowledge base |
| RAGAS faithfulness | >= 0.7 threshold — reinvestigate if below |

## Feature 5 — Six-Layer Guardrail System

Priority: P0 — Critical | Owner: AI Engineering

| Layer | Direction | What it checks |
|---|---|---|
| Format validation | Input | JSON schema, required fields, data types |
| Injection detection | Input | 10 regex patterns — prompt injection, jailbreak attempts |
| Toxicity filter | Input | Harmful or abusive content in alert payload |
| PII auto-redaction | Input | Account numbers, Aadhaar, PAN masked before any LLM call |
| Hallucination detection | Output | Claims without retrievable source flagged |
| Citation audit | Output | Every assertion in final report has a traceable citation |

## Feature 6 — Three-Pillar Observability Stack

Priority: P1 — High | Owner: AI Engineering

Every agent call passes through the @track_agent decorator which emits to three pillars simultaneously — structlog, Prometheus, and LangSmith. Zero monitoring boilerplate exists inside agent code.

| Pillar | Tool | What it provides |
|---|---|---|
| Structured logging | structlog | JSON lines per run, run_id auto-bound, API keys redacted, per-run .jsonl files |
| Metrics | Prometheus + Grafana | Latency histograms, token counters, guardrail hits counter, Grafana alert rules |
| Tracing | LangSmith + OpenTelemetry | Full LangGraph chain trace, prompt/response replay, token cost per agent |

### Prometheus Metrics

| Metric | Type | Tracks |
|---|---|---|
| agent_calls_total | Counter | Success / error / blocked per agent |
| agent_latency_seconds | Histogram | P50 / P95 / P99 per agent |
| llm_tokens_total | Histogram | Token cost per agent per run |
| guardrail_hits_total | Counter | PII / injection / hallucination triggers |

## Feature 7 — Streamlit Dashboard

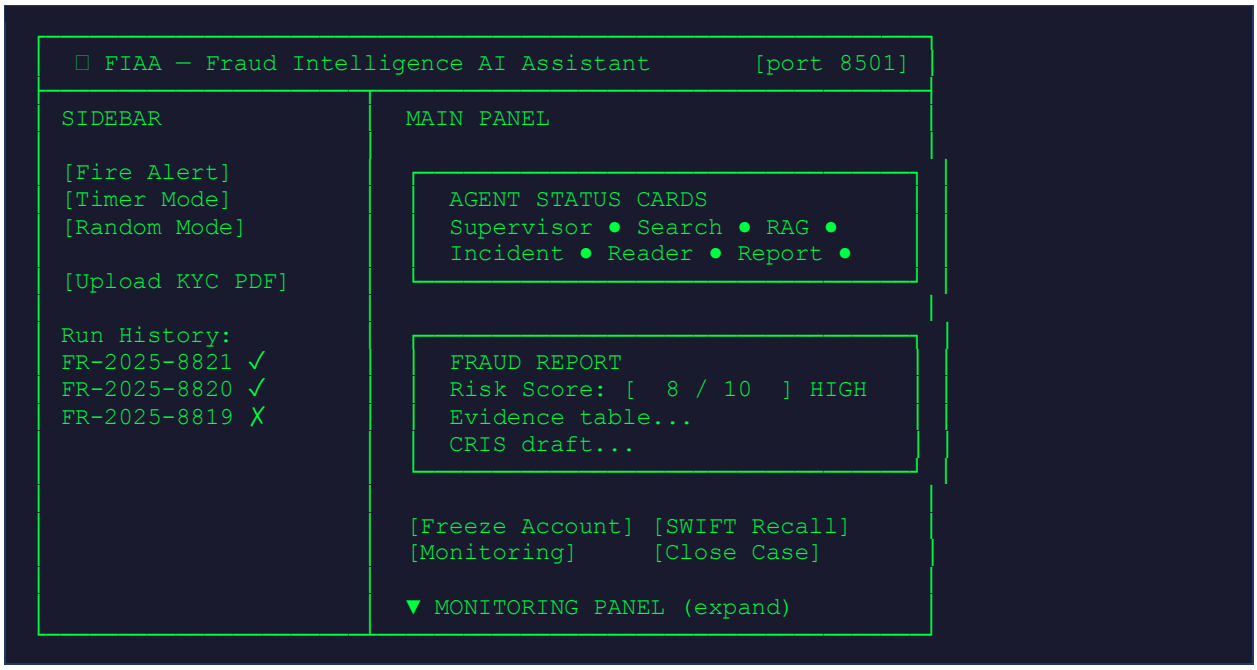
Priority: P1 — High | Owner: AI Engineering

| Panel | Content |
|---|---|
| Live agent status | Real-time cards showing each agent's status — pending / running / complete |
| Fraud report viewer | Formatted report with risk score badge, evidence table, CRIS draft |
| Action buttons | Account Freeze / SWIFT Recall / Enhanced Monitoring / Close Case |
| Monitoring panel | Latency bars, token chart, guardrail log, RAGAS scores |
| Download | Export report as PDF / JSON |
| Alert trigger controls | Manual fire, timer mode, random mode for demonstration |

# 5. UI Mocks & Screen Flows

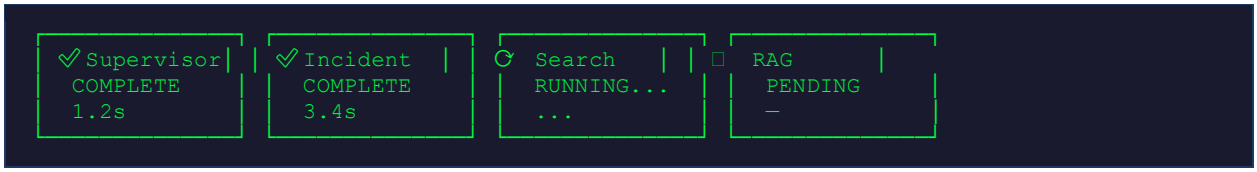
All UI is delivered via Streamlit on port 8501. The following wireframe descriptions define the layout and behaviour of each screen. The dashboard is a single-page application (SPA) with expandable panels.

## 5.1 Dashboard — Main Layout




## 5.2 Agent Status Cards

Live cards update in real-time as each agent runs. States: PENDING (grey) → RUNNING (blue spinner) → COMPLETE (green check) → FAILED (red X).




## 5.3 Fraud Report Panel

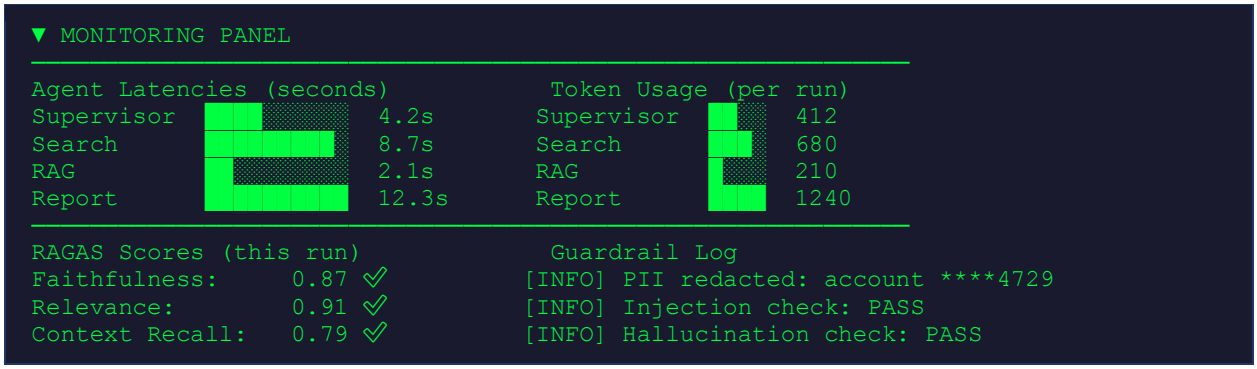
| UI Element | Description | Behaviour |
|---|---|---|
| Risk Score Badge | Large coloured badge — green (1–3), amber (4–6), red (7–10) | Prominently displayed at top of report |
| Evidence Table | Signal, Source, Confidence columns | Expandable rows with source citations |
| CRIS Draft | Pre-filled regulatory summary text | Editable by analyst before submission |
| Recommended Actions | Ordered list of recommended steps | Each step links to action button |
| Timeline | Chronological transaction events | Sortable, filterable |

| UI Element | Description | Behaviour |
|---|---|---|
| Regulatory Deadlines | RBI filing windows, SWIFT window countdown | Colour-coded urgency (green/amber/red) |

## 5.4 Action Buttons

| Button | Action | Confirmation Required |
|---|---|---|
| Freeze Account | Logs action, updates incident status | Yes — modal confirmation |
| SWIFT Recall | Logs case details + timestamp for correspondent bank | Yes — modal with deadline |
| Enhanced Monitoring | Flags account for 30-day enhanced monitoring | No |
| Close Case | Auto-closes with log entry | Yes — confirm close |
| Download Report | Exports PDF / JSON of full report | No |

## 5.5 Monitoring Panel



## 6. Technology Stack

| Layer | Technology | Version | Rationale |
|---|---|---|---|
| Agent orchestration | LangGraph | 0.2+ | Typed shared state + conditional routing + parallel fan-out |
| LLM inference | Groq API — Llama 3.3 70B | latest | 500 tok/s throughput. Fallback chain on 429: 70B→8B→gemma2 |
| Vector database | ChromaDB | 0.5+ | PersistentClient — local, privacy compliant, no cloud |
| Embeddings | sentence-transformers all-MiniLM-L6-v2 | 2.7.0 | CPU only, no API cost, GDPR safe, 384-dim |
| Web search | Tavily API | 0.3+ | Pre-extracted content, relevance-scored, research-optimised |
| API layer | FastAPI + uvicorn | 0.110+ | Async background tasks, rate limiting, Pydantic validation |
| UI | Streamlit | 1.35+ | Live agent cards, monitoring panel, auto-trigger modes |
| Logging | structlog | 24.1+ | JSON lines, run_id binding, secrets auto-redacted |
| Metrics | Prometheus + Grafana | — | Latency histograms, token counters, alerting |
| Tracing | LangSmith + OpenTelemetry | — | Full chain traces, prompt/response replay |
| RAG evaluation | RAGAS | 0.1+ | Faithfulness, relevance, recall scored per run |
| Deployment | Docker Compose | — | 4-service containerised, one-command startup |
| PDF reading | PyMuPDF | — | KYC PDF and account statement text extraction |

# 7. Deployment Architecture

## 7.1 Docker Compose Services

| Service | Image | Ports | Volumes / Config |
|---|---|---|---|
| app | Custom Dockerfile | 8501 (Streamlit), 8001 (FastAPI) | chroma_db:/ logs:/ env_file:.env |
| chroma | chromadb/chroma | 8002 | chroma_db:/chroma/.chroma |
| prometheus | prom/prometheus | 9090 | prometheus.yml:/etc/prometheus/prometheus.yml |
| grafana | grafana/grafana | 3000 | GF_SECURITY_ADMIN_PASSWORD in environment |

## 7.2 One-Command Startup Sequence

| Step | Command | Result |
|---|---|---|
| 1. Configure | cp .env.example .env (edit API keys) | GROQ_API_KEY · TAVILY_API_KEY · LANGCHAIN_API_KEY set |
| 2. Build RAG KB | docker compose run app python -m rag.ingestor | 847 chunks indexed into ChromaDB volume |
| 3. Launch all | docker compose up -d | All 4 services running in background |
| 4. Open UI | http://localhost:8501 | Streamlit dashboard live |
| 5. Test webhook | curl -X POST localhost:8001/api/analyse ... | Pipeline fires, report appears on dashboard |
| 6. View metrics | http://localhost:3000 (admin/admin) | Grafana dashboard with all metric charts |
| 7. View traces | smith.langchain.com → fiaa-fraud project | Full LangGraph chain trace for every run |

## 8. Risks & Mitigations

| Risk | Likelihood | Impact | Mitigation |
|---|---|---|---|
| Groq API rate limit (429) | Medium | High | Fallback chain: 70B → 8B → gemma2-9b. Exponential backoff. |
| LLM hallucination in report | Medium | High | RAGAS faithfulness gate + Evaluator citation audit + output guardrails. |
| PII leakage in output | Low | Critical | Input PII redaction + output PII scan. Zero-tolerance policy. |
| Prompt injection via alert payload | Low | High | 10 regex injection patterns checked on every input. |
| ChromaDB index loss on restart | Low | High | PersistentClient with Docker volume — index survives restarts. |
| Report quality below threshold | Medium | Medium | Reinvestigate loop — Evaluator rejection sends back to Supervisor. |
| RAGAS score below 0.7 | Low | Medium | Grafana alert fires. Human review flagged. Reinvestigate triggered. |